🛒 ReStock AI

A Human-Governed Autonomous Purchasing Officer — the AI handles the volume, the human owns the judgment.

Agentika 2.0 · NaSCon 2026 | Theme: Human + AI — Better Together | Deployed on Clouds by Alternate Futures

Team & submission details

Team name	[FILL: Your team name]
Member 1	[FILL: Full name] · phone & CNIC provided in the Google Form
Member 2	[FILL: Full name] · phone & CNIC provided in the Google Form
Live deployment	https://alternate-restless-forest-3113-github-app.alternatefutures.ai
Source code	https://github.com/chhuzaifa928/Restocker
Demo video (2–3 min)	[FILL: Unlisted YouTube / Google Drive "Anyone with the link" URL]

Please verify this link opens in an incognito window before submitting. No API keys or passwords appear anywhere in this document or the repository.

The problem

Distributors make thousands of restock decisions every week. **Automate all of them** and a single mispriced or oversized order burns the budget. **Do them by hand** and the team drowns in routine approvals. Neither extreme is acceptable — the routine volume needs automation, but the risky calls need human judgment.

The solution

ReStock AI is an AI-driven replenishment officer. A pre-trained **Q-Learning agent** decides how much to reorder for each product, and a **dual safety gate** routes every decision one of three ways:

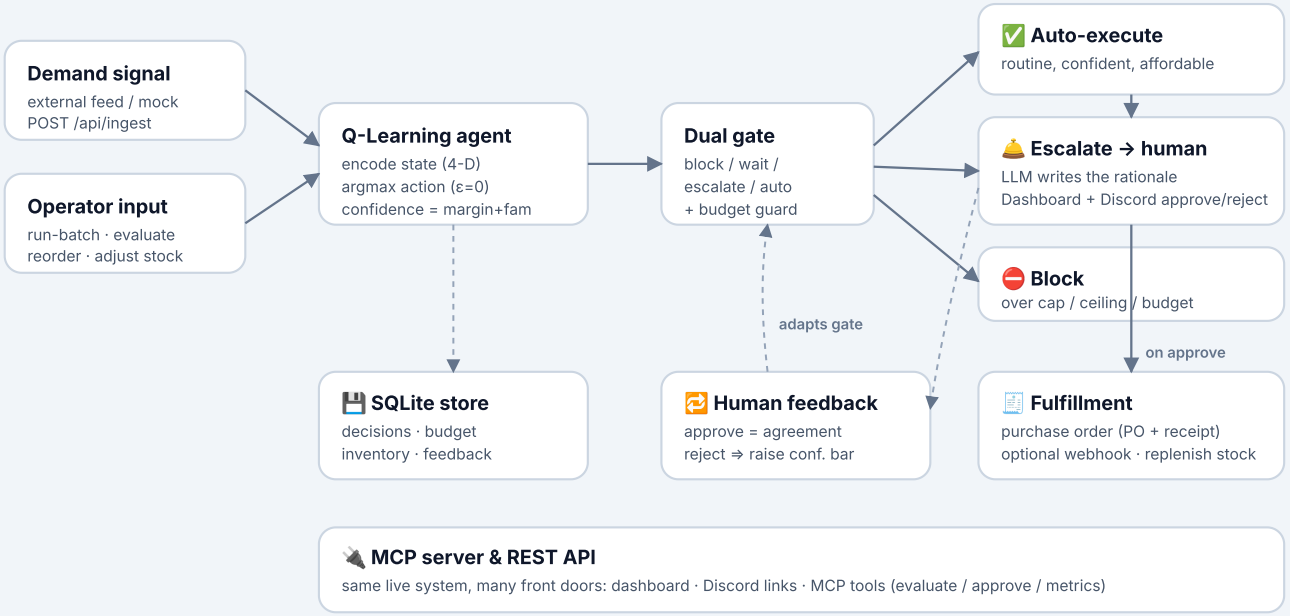
- ✅ **Auto-execute** — routine, confident, affordable orders run without a human.
- 🛎️ **Escalate to a human** — expensive, low-confidence or unfamiliar orders are queued for one-click approval, with a plain-English rationale written by an LLM.
- ⛔ **Block** — orders over the quantity cap, cost ceiling or remaining budget are refused outright.

> **Where the human steps in:** escalated orders wait in the approval queue (and are pushed to Discord with Approve/Reject links). A human approves or rejects; approval issues a real purchase order, rejection voids it *and* makes the gate more cautious. **The LLM only explains — it never decides.**

How it works

ReStock AI — human-governed purchasing pipeline

AI handles the volume · the human owns the judgment · the LLM explains, it does not decide



Key features

Capability	What it does
Q-Learning agent	Encodes each situation into a 4-D state and selects the reorder action (inference only; explainable policy).
Dual safety gate	Auto-execute / escalate / block, with a hard budget guard.
Human-in-the-loop	Approval queue on the dashboard + Discord approve/reject links (approve from your phone).
Real fulfillment	Executed orders raise a purchase order (PO number + receipt), optionally POSTed to a downstream webhook.
Adaptive governance	Human rejections raise the confidence bar so the gate escalates sooner — governance only, the model is never retrained.
Reactive ingestion	A demand signal (external feed or mock) auto-triggers re-evaluation: new demand → decision.
Persistence	SQLite keeps decisions, budget and feedback alive across a redeploy.
MCP server	The same live service is driveable from any MCP client (e.g. Claude Desktop).
Polished UX	Light/dark dashboard, live counters, scenario presets, CSV audit export; 24 automated tests.

Technology

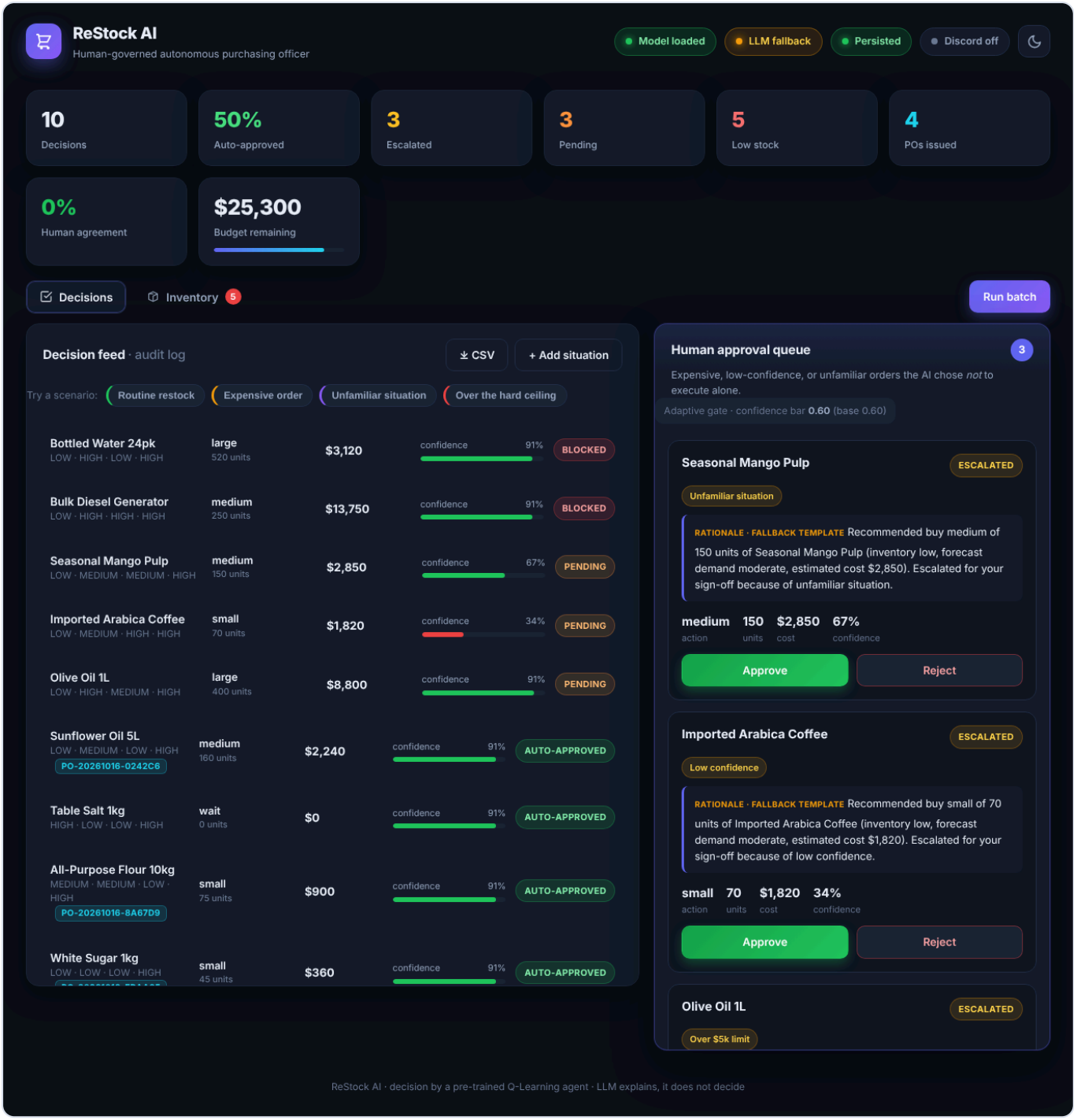
Backend: Python 3.11 · FastAPI · Pydantic · httpx | **Agent:** Q-Learning policy over 81 discretised states | **Storage:** SQLite | **Frontend:** vanilla HTML/CSS/JS single-page dashboard (themeable) | **Integrations:** Discord/Slack webhooks, purchase-order webhook, Model Context Protocol | **Deploy:** Docker on Clouds by Alternate Futures (GitHub repo → container, $PORT-aware).

How it meets the judging rubric

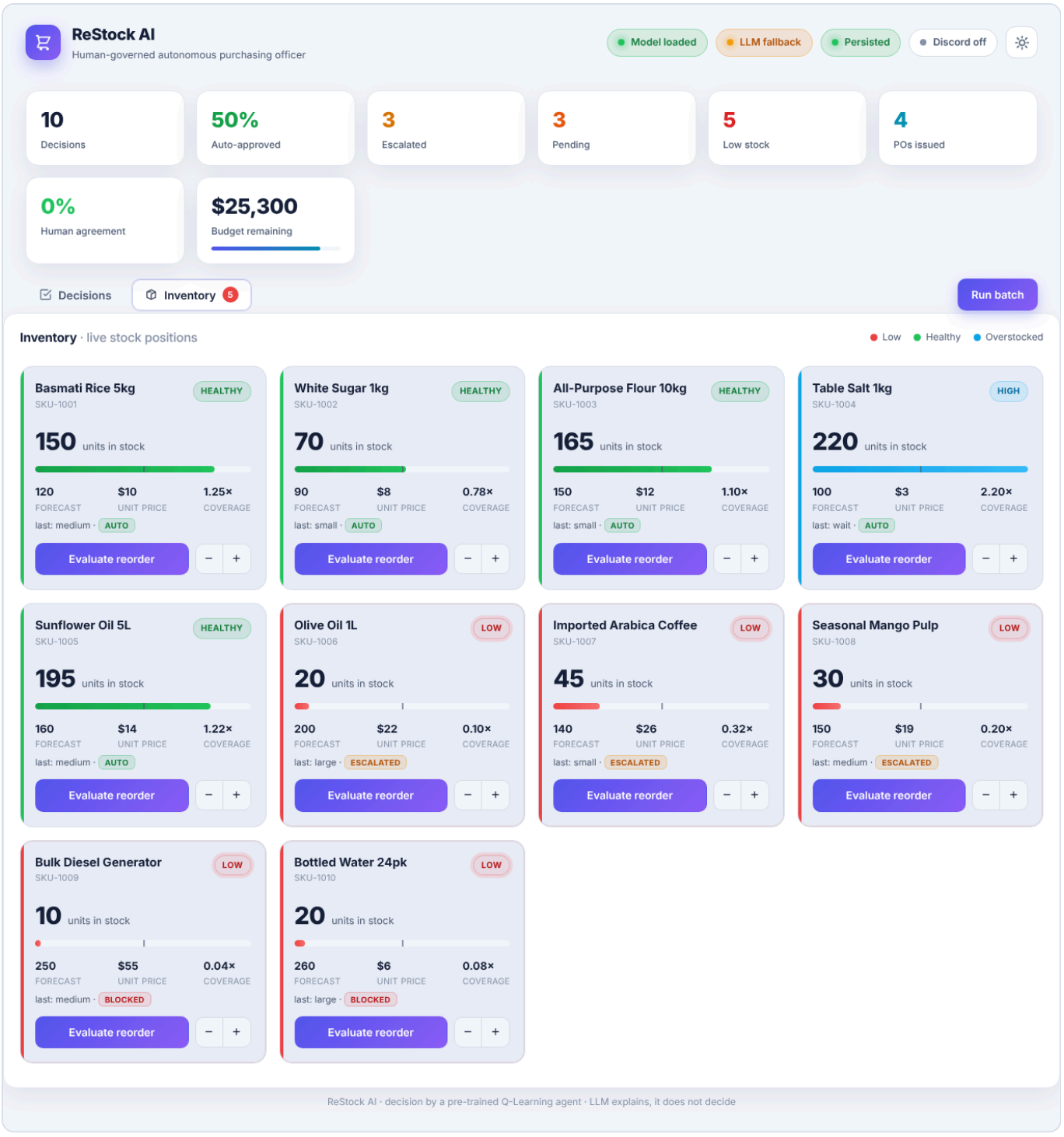
Criterion	In ReStock AI
Technical execution & reliability	End-to-end working, graceful degradation everywhere, persistence, 24 passing tests.
Usefulness & real-world impact	Real replenishment problem, budget control, actual purchase-order output.
Theme alignment	Human + AI: the AI auto-handles volume; the human approves the risky orders.
Creativity & uniqueness	Dual gate + adaptive governance + reactive ingestion; the LLM explains, not decides.
Automation depth & integration	Discord approvals + fulfillment webhook + MCP server + demand ingestion.
User experience & presentation	Clean light/dark dashboard, live metrics, toasts, one-click demo scenarios.

Screenshots

Decisions view (dark theme) — audit-log feed + human approval queue.



Inventory view (light theme) — live stock positions; executed orders replenish stock.



An honest note on the model

The Q-table in this build is a **synthesised, explainable policy** covering all 81 states — not a trained one — and it is a drop-in artifact: a notebook-exported Q-table with the same schema plugs in unchanged. The feedback loop adapts the *gate*, never the learned values, and the LLM only writes rationales. We chose explainability and a reliable live demo, and we are transparent about it.